

Fields And Methods In Java Classes

Generated Aug 19, 2026 8:54 AM

What We'll Learn

By the end of this lesson, students can:

- Explain the purpose of fields and methods in a Java class.
- Declare fields using appropriate data types.
- Create methods that read or modify an object's fields.
- Create methods with parameters.
- Create methods that return values.
- Access fields and call methods using an object.

Before We Start

A class can describe what an object knows and what it can do.

Consider a bank account.

- What information should a bank account store?
- What actions should a bank account perform?
- Which parts should be represented as fields?
- Which parts should be represented as methods?

Let's Understand

Fields

A field is a variable declared inside a class but outside its methods.

Fields store information about an object. They describe the object's characteristics or current condition.

```
class Student {  
    String name;  
    int age;  
    double average;  
}
```

The Student class has three fields:

- name
- age
- average

Every Student object receives its own copy of these fields.

```
Student student1 = new Student();  
Student student2 = new Student();  
  
student1.name = "Angela";  
student2.name = "Joshua";
```

Changing the name of student1 does not change the name of student2.

Default Field Values

When an object is created, Java automatically gives its fields default values if no value has been assigned.

Data Type	Default Value
int	0
double	0.0
boolean	false
char	Empty character
String	null

Consider the following class:

```
class Product {  
    String name;  
    double price;  
    boolean available;  
}
```

If a Product object is created without assigning values, its fields initially contain:

```
name = null  
price = 0.0  
available = false
```

Methods

A method is a named block of code that performs a specific task.

Methods represent the actions or behaviors of an object.

```
class Student {  
    String name;  
  
    void introduce() {  
        System.out.println("Hello! My name is " + name + ".");  
    }  
}
```

The introduce() method uses the object's name field.

Parts Of A Method

Study the following method:

```
void displayMessage() {  
    System.out.println("Welcome to Java!");  
}
```

Its parts are:

- void – the method does not return a value.
- displayMessage – the name of the method.
- () – contains the method's parameters, if any.
- { } – contains the statements performed by the method.

Methods That Read Fields

A method can access and display the fields of its object.

```
class Book {
    String title;
    String author;

    void displayInformation() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
    }
}
```

When an object calls `displayInformation()`, the method uses the fields belonging to that object.

Methods That Modify Fields

A method can change the values stored in an object's fields.

```
class Lamp {
    boolean isOn;

    void turnOn() {
        isOn = true;
    }

    void turnOff() {
        isOn = false;
    }
}
```

The methods `turnOn()` and `turnOff()` modify the `isOn` field.

Methods With Parameters

A parameter allows a method to receive information when it is called.

```
class GameCharacter {
    int health;

    void receiveDamage(int damage) {
        health = health - damage;
    }
}
```

In this method, `damage` is a parameter.

The method can be called by providing an argument:

```
player.receiveDamage(20);
```

The value `20` is passed to the `damage` parameter.

Methods That Return Values

A method can calculate and return a value to the statement that called it.

```
class Rectangle {  
    double length;  
    double width;  
  
    double calculateArea() {  
        return length * width;  
    }  
}
```

The method uses `double` instead of `void` because it returns a decimal value.

The `return` statement sends the result back to the calling statement.

```
double area = rectangle.calculateArea();
```

A returning method can also be used directly inside an output statement:

```
System.out.println(rectangle.calculateArea());
```

Local Variables

A variable declared inside a method is called a local variable.

```
class Rectangle {  
    double length;  
    double width;  
  
    double calculateArea() {  
        double area = length * width;  
        return area;  
    }  
}
```

In this example:

- `length` and `width` are fields.
- `area` is a local variable.

A local variable can only be used inside the method where it was declared.

Fields And Methods Working Together

Fields store the object's current data, while methods use or modify that data.

```
class BankAccount {  
    String ownerName;
```

```
double balance;

void deposit(double amount) {
    balance = balance + amount;
}

void displayBalance() {
    System.out.println(ownerName + "'s balance: PHP " + balance);
}
}
```

The balance field stores the current balance. The deposit() method changes it, while displayBalance() reads and displays it.

Let's Practice

Activity 1: Identify The Fields And Methods

Study the following class:

```
class Cellphone {  
    String brand;  
    int batteryLevel;  
  
    void charge(int amount) {  
        batteryLevel = batteryLevel + amount;  
    }  
  
    void displayBattery() {  
        System.out.println("Battery: " + batteryLevel + "%");  
    }  
}
```

Answer the following questions:

1. What are the fields of the class?
2. What are the methods of the class?
3. Which method has a parameter?
4. What is the name of the parameter?
5. Which field is modified by the charge() method?

Activity 2: Field Or Local Variable

Identify whether each variable is a field or a local variable.

```
class Circle {  
    double radius;  
  
    double calculateArea() {  
        double area = 3.14159 * radius * radius;  
        return area;  
    }  
}
```

Classify the following:

1. radius
2. area

Explain why each variable belongs to its classification.

Activity 3: Trace The Program

Study the following program:

```

class Counter {
    int value;

    void increase(int amount) {
        value = value + amount;
    }

    int getValue() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) {
        Counter counter = new Counter();

        counter.value = 5;
        counter.increase(3);
        counter.increase(2);

        System.out.println(counter.getValue());
    }
}

```

Write the exact output of the program.

Activity 4: Complete The Methods

Complete the missing parts of the following class:

```

class Rectangle {
    double length;
    double width;

    double calculateArea() {
        return _____;
    }

    void displayDimensions() {
        System.out.println("Length: " + _____);
        System.out.println("Width: " + _____);
    }
}

```

Activity 5: Create Fields And Methods

Create a BankAccount class with the following fields:

- accountName
- balance

Add the following methods:

- `deposit(double amount)` – adds the amount to the balance.
- `withdraw(double amount)` – subtracts the amount from the balance.
- `displayBalance()` – displays the account name and current balance.

Inside the `main()` method:

1. Create one `BankAccount` object.
2. Assign an account name and starting balance.
3. Deposit an amount.
4. Withdraw an amount.
5. Display the final balance.